

Experiment Runner: a Tool for the Automatic Orchestration of Experiments Targeting Software Systems

Max Karsten, Andrei Calin Dragomir, Radu Apsan, Vincenzo Stoico, Ivano Malavolta

PII: S0167-6423(25)00154-6
DOI: <https://doi.org/10.1016/j.scico.2025.103415>
Reference: SCICO 103415



To appear in: *Science of Computer Programming*

Received date: 21 May 2025
Revised date: 12 November 2025
Accepted date: 12 November 2025

Please cite this article as: Max Karsten, Andrei Calin Dragomir, Radu Apsan, Vincenzo Stoico, Ivano Malavolta, Experiment Runner: a Tool for the Automatic Orchestration of Experiments Targeting Software Systems, *Science of Computer Programming* (2025), doi: <https://doi.org/10.1016/j.scico.2025.103415>

This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Experiment Runner: a Tool for the Automatic Orchestration of Experiments Targeting Software Systems

Max Karsten

Vrije Universiteit Amsterdam, the Netherlands

Andrei Calin Dragomir

Vrije Universiteit Amsterdam, the Netherlands

Radu Apsan

Vrije Universiteit Amsterdam, the Netherlands

Vincenzo Stoico

Vrije Universiteit Amsterdam, the Netherlands

Ivano Malavolta

Vrije Universiteit Amsterdam, the Netherlands

Abstract

Experiment Runner (ER) is a Python framework for the (semi-) automatic orchestration of measurement-based experiments targeting software systems. ER allows users to (i) programmatically define an experiment in terms of its factors, treatments, subjects, etc., (ii) start, stop, pause, and resume an experiment with no data loss, (iii) integrate third-party software- or hardware-profilers, (iv) execute an experiment either in automatic mode or semi-automatic mode, and (v) define custom callbacks that are automatically called by ER when needed. ER has been used in tens of empirical studies in the Software Engineering area. In this paper, we describe the main features of ER, its software architecture, a simple experiment executed by ER, and three scientific studies where it has been used successfully.

Keywords: Empirical Software Engineering, Experiment Orchestration,

Email addresses: `m.h2.karsten@student.vu.nl` (Max Karsten),
`a.dragomir@student.vu.nl` (Andrei Calin Dragomir), `r.apsan@vu.nl` (Radu Apsan),
`v.stoico@vu.nl` (Vincenzo Stoico), `i.malavolta@vu.nl` (Ivano Malavolta)

Benchmarking

Metadata

Table 1: Code metadata

Nr.	Code metadata description	Code metadata
C1	Current code version	V1.0.1
C2	Permanent link to code/repository used for this code version	https://github.com/S2-group/experiment-runner
C3	Permanent link to Reproducible Capsule	https://doi.org/10.5281/zenodo.15430328
C4	Legal Code License	MIT license (MIT)
C5	Code versioning system used	Git/GitHub
C6	Software code languages, tools, and services used	Python, Shell
C7	Compilation requirements, operating environments and dependencies	Required Python libraries: pandas, psutil, tabulate, dill, jsonpickle
C8	If available, link to developer documentation/manual	https://github.com/S2-group/experiment-runner/wiki
C9	Support email for questions	i.malavolta@vu.nl

1. Motivation and significance

Empirical software engineering is founded on rigorous, repeatable, and data-driven experimentation [1]. Nonetheless, the practical challenges of conducting high-quality, replicable experiments continue to place a heavy burden on researchers. The setup, coordination, and execution of experiments (especially those involving complex software configurations, randomized treatments, and long-running measurements) are still often carried out manually or using ad-hoc scripts, leading to low reproducibility, high setup costs, and limited traceability.

Experiment Runner (ER) is a **domain-agnostic framework that enables streamlining the orchestration of measurement-based experiments in a (semi-) automated fashion**. ER supports randomized treatment execution, repeated runs under varying conditions, and the ability to pause and resume experiments without data loss. Such features help in reducing bias and improving results' reliability while also aligning with best practices in empirical software engineering [1, 2].

By abstracting away execution control logic, ER allows researchers to focus on designing their studies rather than managing their low-level details and logistics. ER has already been used in more than 50 empirical studies

involving performance profiling, benchmarking, and resource usage analysis and is especially useful in long-running or failure-prone environments [3, 4]. The extensive [use](#) (and independent verification via the peer review process) of ER serves as qualitative evidence of ER’s reliability, effectiveness, and utility in diverse contexts. Unlike domain-specific tools (*e.g.*, Robot Runner for robotics [5]), ER emphasizes flexible orchestration over measurement instrumentation, still allowing users to implement their experiment-specific business logic in the tool.

In practice, users define their experimental plan in Python, specifying parameters such as the number of repetitions of each trial, the scheduling constraints, and optional setup/teardown logic. ER then manages the execution schedule, handles interruptions gracefully, and logs execution metadata consistently. Users can plug in their measurement tools or experiment-specific business logic, making the system flexible for a broad range of scientific domains.

1.1. Common Use Cases

Accurate profiling of software systems faces numerous challenges that can undermine the reliability of experimental results. ER is designed to alleviate this burden for users. It lets users select different tools for any metric and granularity, and run them concurrently. For instance, users can employ separate tools to monitor energy usage at both the application and function levels while simultaneously profiling CPU and memory consumption.

[Adding or removing tools typically incurs development effort and time, and involves a learning curve.](#) ER addresses this by providing a unified interface that supports a wide range of profiling tools. This consistency enables researchers to compare different configurations efficiently, allowing them to identify and verify the optimal tool with minimal overhead. As an example, choosing a tool for power consumption measurement can be especially tricky because the measurement instruments themselves may affect performance. ER makes it easy to swap or evaluate such tools against one another. ER can measure its own power consumption, offering deeper insight into any side effects introduced by the profiling process itself.

For a concrete example, ER can be used to set up an experiment in which several optimization levels of a compiler with respect to power consumption. Without ER, this experiment poses a number of challenges, mainly around creating an environment where the compiled binaries are executed and measured using a profiler. ER simplifies this process by supplying standard tools for setting up these types of experiments. Its callback-based architecture offers a convenient way to compile the binaries first and then run them in

separate steps, cleanly separating measurement from setup. After the experiment finishes, the run-table model aggregates the detailed logs, providing immediate summarisation for the large data sets generated by many profilers. A further advantage is that modifications to the experiment rarely require extensive rewrites of the code. If you want an additional dataset with, for instance, memory consumption data, the `ps` profiler can be used on Linux. Likewise, if you prefer to measure power consumption using external hardware, ER also supports a range of compatible devices.

2. Software description

In this section, we provide an overview of ER. Specifically, we present the key features of ER in Section 2.1, then we describe the architecture of ER in Section 2.2, and finally, we elaborate on the implementation details of one of the key functions of ER, *i.e.*, the `do_experiment` in Section 2.3, together with the main design decisions we took while developing it.

2.1. Software functionalities

ER aims to provide a generic, platform-agnostic framework to collect metrics on a target piece of software. To achieve this, ER models the experimentation process as a discrete set of sequential events, each triggering a user-defined callback. This set of callbacks constitutes the user-defined configuration and provides ER with the desired set of actions that manage the target program and its measurement. To facilitate this, ER offers several key features: the run table model, data persistence, crash recovery, different operational types, progress logging, and profiling plugins.

The run table model allows the user to define the layout of their results as a Python class; ER then does the work to execute all runs of the experiment and generate a corresponding output file. To make this functionality more robust, ER persists data to the run table file after each experiment run. In the event of a crash, ER uses this persisted data to resume the experiment at the point it left off automatically. Lastly, while ER is target- and profiler-agnostic, several profiler plugins are provided as the user defines the interaction with both of these interfaces. These plugins offer a convenient and consistent interface to a wide range of common profiling tools such as Linux `ps` [6], macOS `powermetrics` [7], and physical devices such as WattsUpPro [8]. This makes collecting data on any target machine much simpler as ER takes care of managing the creation and parsing of (typically tool-specific) logs containing the raw data.

ER provides the users with a log that includes the execution progress of the experiment and each run, and the status of the experiment (executing,

completed, or if there are any errors). This log is crucial to users to *reproduce* experiments, *fix* runs, and *restart* them in case of any errors. ER also provides two operational modes: (i) *automatic*, and (ii) *semi-automatic*, which allows the user to have fine-grained control over how their experiment progresses with minimal configuration.

2.2. Software architecture

Figure 1 presents an overview of the software architecture of Experiment Runner. The **Runner configuration** is the main component used by the user to interact with the ER system. The Runner configuration is a single Python file that defines a class containing all experiment-specific business logic. Among several configuration parameters (which is described later in this document), the Runner configuration class follows the *inversion of control* principle. Specifically, the user can specify 10 (optionally 11) abstract operations that act as *callback* functions in ER. By referring to Figure 2, the callback functions currently provided by ER are:

1. *Create run table*: called once if no existing run table is found. Enumerates all experiment combinations, creates a table format, and files to persist results.
2. *Before experiment*: it is executed before the experiment is started. This callback is typically used for configuring the machine(s) where the experiment is executed, setting up environmental variables, etc..
3. *Before run*: this callback is started before the execution of each individual run of the experiment. This callback is typically used to configure the experiment according to the treatment applied in the current run.
4. *Start run*: this function executes an experimental run (*e.g.*, for launching an Android app, starting a robotic mission).
5. *Start measurement*: it is called to start the profiler(s) in order to collect measures from the target machines(s) or software.
6. *Interaction*: it is called right after the measurement starts and it aims to define and execute some possible actions to be executed during the experimental run (*e.g.*, executing a specific workload in case of benchmarking experiments).
7. *Stop measurement*: it stops the profiler(s).
8. *Stop run*: it stops the current run. This callback is typically used for cleanup activities in-between experimental runs.
9. *Populate run data*: it allows the user to read in data collected from the profiler(s) and aggregate the collected results for the progress manager.
10. *After experiment*: it is executed at the end of the experiment, after all experimental runs are executed.

11. *(Optional) Continue*: it allows the user to manually resume the experiment after a run. Note: it is available only in the semi-automatic mode, where the next run begins only if the user explicitly indicates they want to continue.

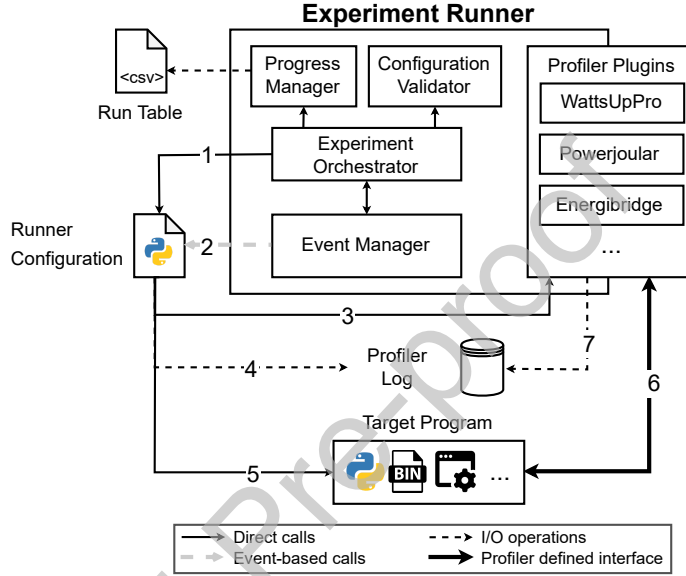


Figure 1: Overview of the software architecture of Experiment Runner

The user interacts mainly with the target program, a set of profilers, or an external measurement system, and any data generated by the profiler(s) as shown in connections 3, 4, and 7 in Figure 1. The experiment orchestrator calls the configuration file to initialize the system and implements the callback system, triggering user-defined callbacks via the event manager. For the interested reader, several example configurations can be found in the GitHub repository of ER under the “examples” folder.

The **Experiment orchestrator** component is the core of the system. It provides features such as path validation, file system setup for storing the results of the experiment, crash handling, and initialization. In the initialization process, the **Configuration validator** is used to generate the *run table*. The run table serves as both a plan containing the levels of the various independent variables for each run and a model for the aggregated results. Internally, each run of the experiment is represented by a tuple composed of (i) a unique identifier, and (ii) the status of the run. The user only provides

a definition for this table and populates the fields after each run. An experiment configuration is identified by an MD5 hash, which the orchestrator also uses to restart the experiment, for example in case of crash recovery. When an existing set of results is found, the orchestrator can deduce if results must still be collected based on the state of this table, thereby recovering from a crash.

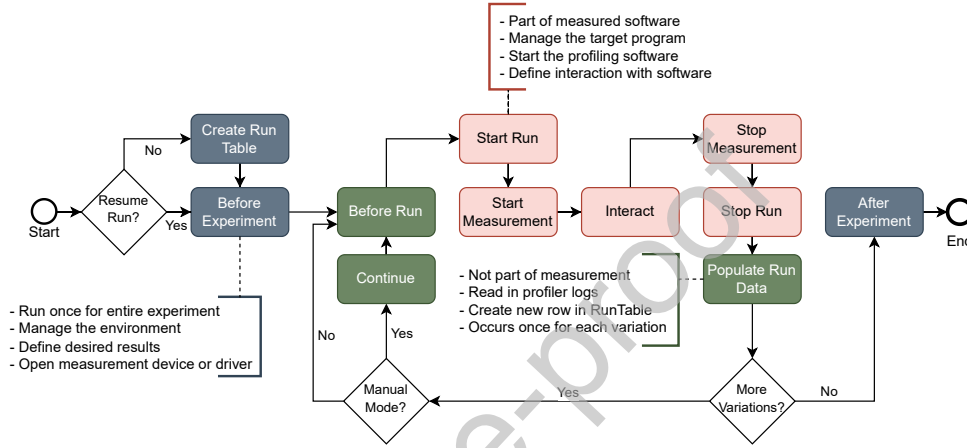


Figure 2: Overview sequence of events that comprise the experiment-runner process.

To start the experiments, the orchestrator calls the before experiment callback to set up any state, as shown in connection 1 in Figure 1, then the experiment variations are run sequentially by starting an instance of the `RunController` class, which calls the rest of the callbacks defined in the user configuration, and subscribed to using the **Event Manager**. Although fairly simple in nature, the event manager is what implements the event subscription functionality that ER is based on. The orchestrator also supports the ability to measure the overall power consumption of the system during operation through the integration of the EnergiBridge [9] profiler. There are two modes of operation supported by the orchestrator: (i) *automatic*, where all experiments are run back-to-back and (ii) *semi-automatic*, where a `continue` callback can be defined that, for instance, allows a user to manually start the experiment for each run (this feature proved works particularly well for experiments with a physical component or those in which physical user interaction is key).

After the execution of each experimental run, the orchestrator calls the **Progress manager**. This subsystem manages the persistence of run table data in different formats. By default, the run table is output in CSV format, but JSON is also supported. The output process involves adding a row to

the existing table using the raw data collected by the used profiler(s). The user aggregates these results and provides a new entry for each column they defined in the run-table model.

ER provides a **plugin-based subsystem** where user- or community-defined profilers can be integrated into ER without any changes to the rest of the framework. These plugins wrap a set of different profilers (*e.g.*, for energy, CPU usage, etc.) in a consistent interface, allowing them to be easily integrated into ER-based experiments. Since many of these profilers are in the form of command-line interfaces, the plugins handle the starting and termination of these processes, as well as methods to read the created logs into Python data structures (*e.g.*, dictionaries). The ER plugins subsystem defines two interfaces, one for CLI plugins and one for hardware meters plugins. The CLI interface defines a start and stop method and allows all the parameters passed to the profiler to be validated and the output to be collected. For hardware meter plugins, similar methods are provided: `open_device` and `close_device` as well as a `log` method that outputs the raw data produced by the hardware profiler into a file. This interface mimics how many device drivers commonly work and automatically handles much of the state in addition to abstracting the interface provided by the driver. An overview of the plugins we developed as of May 2025 can be found in Table 2.

Table 2: Plugins available in Experiment Runner as of September 2025

Plugin	Supported OS	Provided Metrics
CodeCarbon [10]	Windows, Linux, macOS	Carbon emissions
powermetrics [7]	macOS	Power consumption, process statistics
powerletrics [11]	Linux	Power consumption, process statistics
Picolog CM3 [12]	Windows, Linux, macOS	External device power consumption
WattsUpPro [8]	macOS, Linux	External device power consumption
PowerJoular [13]	Linux	Power consumption
Nvidia Management Library [14]	Windows, Linux	GPU device statistics (power, utilization, state, ect.)
ps [6]	Linux	Process statistics
EnergiBridge [9]	Windows, Linux, macOS	Power consumption

2.3. Sample code snippets analysis

As an illustrative example of the internal functioning of ER, Listing 1 shows the implementation of its `do_experiment` function. This function is

defined in the `ExperimentController` class of the Experiment Orchestrator module. As described in the previous section, the orchestrator is responsible for interconnecting the different subsystems and initializing the experiment process. This function is called after the run table has been validated and crash recovery has possibly taken place. In line 6 of the listing, we see the call to the user-defined “Before experiment” callback. In ER, the `RunnerEvents` object contains references to the different callbacks created by the user in the runner configuration. Next, on line 9 in the listing, the experimental runs defined in the run table are iterated through, skipping the ones that are already completed. On line 14, ER calls another user-defined callback, *i.e.*, the “before run” one. Then, on lines 16–22, the rest of the callbacks are handed to the event handling system by creating a `RunController` class and using a Python multiprocessing module to start it in a separate process. After the `RunController` process has terminated, thus completing one experimental run, an optional cool-down period is triggered (lines 24–27), followed by an optional call to the continue event that implements the semi-automatic mode (lines 29–30). After all experimental runs are complete, the final “after experiment” callback is called (line 36).

```

1 def do_experiment(self):
2     output.console_log_OK("Experiment setup completed...")
3
4     # -- Before experiment
5     output.console_log_WARNING("Calling before_experiment config hook")
6     EventSubscriptionController.raise_event(RunnerEvents.BEFORE_EXPERIMENT)
7
8     # -- Experiment
9     for current_run in self.run_table:
10         if current_run['_done'] == RunProgress.DONE:
11             continue
12
13         output.console_log_WARNING("Calling before_run config hook")
14         EventSubscriptionController.raise_event(RunnerEvents.BEFORE_RUN)
15
16         run_controller = RunController(current_run, self.config, (self.
17 run_table.index(current_run) + 1), len(self.run_table))
18         perform_run = multiprocessing.Process(
19             target=run_controller.do_run,
20             args=[]
21         )
22         perform_run.start()
23         perform_run.join()
24
25         time_btwn_runs = self.config.time_between_runs_in_ms
26         if time_btwn_runs > 0:
27             output.console_log_bold(f"Run fully ended, waiting for: {
28 time_btwn_runs}ms == {time_btwn_runs / 1000}s")
29             time.sleep(time_btwn_runs / 1000)
30
31         if self.config.operation_type is OperationType.SEMI:
32             EventSubscriptionController.raise_event(RunnerEvents.CONTINUE)
33
34     output.console_log_OK("Experiment completed...")

```

```

33
34 # -- After experiment
35 output.console_log_WARNING("Calling after_experiment config hook")
36 EventSubscriptionController.raise_event(RunnerEvents.AFTER_EXPERIMENT)

```

Listing 1: The do_experiment function from ExperimentController.py

2.4. Plugin Authoring

As mentioned before, ER provides a plugin-based subsystem where profiling tools can be encapsulated behind a uniform interface. To support reproducibility and maintainability, contributors are encouraged to follow a few guidelines when authoring new plugins:

- *Single responsibility*: each plugin should abstract a single tool. If multiple tools are needed, they should be implemented as separate plugins.
- *Reuse of existing constructs*: new plugins should leverage the existing base classes and interfaces found in `Plugins/Profilers/DataSource.py`, ensuring a uniform interaction model across plugins.
- *Concise interface*: plugins should implement the common life-cycle methods of existing plugins (`start`, `stop`, `parse_log`, etc.) so that they can be seamlessly integrated into ER experiments.
- *Examples*: new plugins must include a minimal working example in a dedicated `examples/` directory, together with a `RunnerConfig.py` file showing how the plugin can be used.
- *Documentation*: each new plugin must be accompanied by a `README.md` that briefly describes the tool, its intended purpose, and any dependencies required for execution.

Interfaces. At the core of the plugin system is the abstract `DataSource` class, which defines the minimal contract for any profiling tool: `start`, `stop`, `parse_log`, `supported_platforms`, and `source_name`. Each plugin must extend this class (directly or indirectly), thereby ensuring that all plugins can be managed uniformly by the experiment orchestrator.

Two specializations of the `DataSource` class are provided to simplify development:

- **CLISource**: designed for command-line tools. It automatically validates whether the executable is available on the system, manages process creation/termination, and checks for runtime errors. Plugin authors mainly define the expected parameters (`ParameterDict`), the `source_name`, and parsing logic for the tool's output.

- **DeviceSource:** designed for hardware profilers (e.g., external power meters). It abstracts device discovery (`list_devices`), initialization (`open_device`), and logging via background threads. Plugin authors provide implementations for device-specific setup, logging, and shut-down.

These abstractions allow plugin authors to focus on wrapping the profiler’s functionality rather than reimplementing boilerplate process or device management. For example, implementing a new CLI-based energy profiler typically requires less than 50 lines of code when extending `CLISource`.

Contribution workflow. This minimal authoring workflow ensures that plugins remain consistent, easy to test, and accessible for both novice and advanced users. Further documentation and templates for plugin development are available in the GitHub Wiki of the project ¹. We welcome and appreciate community contributions, which play a key role in extending the coverage of supported profilers and strengthening the long-term sustainability of ER.

3. Illustrative examples

3.1. Hello World Experiment

This section presents a simple experiment showcasing the basic functionalities of ER. Listing 2 shows the Python configuration of such experiment. The file is structured as a set of user-defined fields and 10 functions, one for each phase in the experiment (as shown in the previous section). In the following, we explore all fields and functions of the example experiment.

Python imports (lines 1-14). Lines 1-12 specify the libraries needed to execute ER. Below line 12, the user can add any other packages they might require for running the experiment. A good example of package that can be imported here is `pandas` as this package can be used to do some early data processing.

User Specific Config (lines 15-33). Line 15 defines the `RunnerConfig` Python class, which contains all user-defined fields and functions. Line 16 sets the root directory of the experiment. It is the reference location for all relevant files produced and read by ER during an experiment. User-defined fields start from line 18. In this section of the file, the user defines the `name` of the experiment, the location where the experiment results should be exported to (`results_output_path`), the `operation_type`, and

¹<https://github.com/S2-group/experiment-runner/wiki>

the `time_between_runs_in_ms`. The `operation_type` parameter can be instantiated with two values (`OperationType.AUTO` and `OperationType.SEMI`), and controls what happens after a run is finished, and ER waits for the time chosen in the `time_between_runs_in_ms` parameter. ER provides the `OperationType.AUTO` option to continue the experiment automatically after a specified waiting period. With the `OperationType.SEMI` option, the experiment pauses after waiting for the chosen time. The experiment is only resumed after the callback to `RunnerEvents.CONTINUE` returns.

Initialization (lines 34-53). This function is called at the beginning of the experiment. It is meant to (i) link the callbacks defined in the remainder of the `RunnerConfig` class to the ER events described in the previous section (lines 39-49) and (ii) initialize the reference to the run table (line 50).

create_run_table_model (lines 54-69). In this function, the user defines the factors, treatments, repetitions, and data columns that are going to be considered during the experiment. In our example, we target three implementations of the Fibonacci sequence algorithm and benchmark them. The factors are defined using the `FactorModel` using the factor name and the treatments for that factor. In our example, the implementation (`fib_type`) represents the main factor of the experiment, with three possible treatments, *i.e.*, `iter`, `mem`, and `rec`, one for each implementation to be benchmarked. We also test the Fibonacci program with different inputs. Therefore, the input size corresponds to `factor2` with values 10, 35, 40, 5000, and 10000.

With the current configuration, ER runs all combinations of treatments: `iter-10`, `iter-35`, `iter-40`, `iter-5000`, `iter-10000`, `mem-10`, `mem-35`, `mem-40`, `mem-5000`, `mem-10000`, `rec-10`, `rec-35`, `rec-40`, `rec-5000`, and `rec-10000`. If the user wants to exclude a particular combination of factors from the experiment, then the `exclude_combinations` field of the run table model can be instantiated accordingly. For example, in this experiment we are discarding (i) all combinations with problem size 10 (line 62), (ii) the combinations with implementation `rec` and problem sizes 5000 and 10000 (line 63), (iii) the combinations with implementations `mem` and `iter` and problem sizes 35 and 40 (line 64). Accordingly, ER is going to run the following combinations: `iter-5000`, `iter-10000`, `mem-5000`, `mem-10000`, `rec-35`, and `rec-40`.

The `repetitions` field represents the number of times each combination of treatments is repeated by ER. This field is useful in experiments involving physical or non-deterministic processes (*e.g.*, experiments involving energy measures [2]), and ensures that the data extracted is representative (given that the number of repetitions is large enough).

The `data_columns` field defines the names of the columns where the collected measures are stored.

Unused callbacks. Our experiment does not use the `before_experiment`

(lines 71-74), `before_run` (lines 76-79), `start_run` (lines 81-85), `interact` (lines 97-99), `stop_run` (lines 106-108), and `after_experiment` (lines 122-125) callbacks. As a result, they are left with a `pass` instruction.

start_measurement (lines 87-95). This callback is dedicated to starting the profiler that is going to be collecting data. The profiler is also used to start the Fibonacci program, so we start the measurement and the program in the `start_measurement` function. Using the `context.execute_run` variable, we get the treatment combination that is currently under test (lines 88 and 89). We then set `EnergiBridge` as the profiler using the `self.profiler` variable (lines 92-93). The `EnergiBridge` profiler takes in two arguments: the target program (*i.e.*, program under test) and the output file name. We start the profiler once it is instantiated (line 95).

stop_measurement (lines 101-103). This callback is responsible for stopping the measurement. In our case, `EnergiBridge` is going to stop automatically when the target program finishes. Therefore, we do not want to stop the measurement early, so we wait for the process to finish running.

populate_run_data (lines 110-120). In this callback, the data measured by `EnergiBridge` is collected (lines 115 and 116) and sent to be stored in the summary file by `ER` (lines 118-120). Here is a great opportunity for the user to do early data processing, such as creating summary statistics, detecting outliers, etc.. Note that the keys in the `dict` object returned have to be the same as the data columns defined in line 67.

```

1 from EventManager.Models.RunnerEvents import RunnerEvents
2 from EventManager.EventSubscriptionController import
  EventSubscriptionController
3 from ConfigValidator.Config.Models.RunTableModel import RunTableModel
4 from ConfigValidator.Config.Models.FactorModel import FactorModel
5 from ConfigValidator.Config.Models.RunnerContext import RunnerContext
6 from ConfigValidator.Config.Models.OperationType import OperationType
7 from ProgressManager.Output.OutputProcedure import OutputProcedure as output
8 from Plugins.Profilers.EnergiBridge import EnergiBridge
9
10 from typing import Dict, List, Any, Optional
11 from pathlib import Path
12 from os.path import dirname, realpath
13
14
15 class RunnerConfig:
16     ROOT_DIR = Path(dirname(realpath(__file__)))
17
18     # ===== USER SPECIFIC CONFIG =====
19     """The name of the experiment."""
20     name: str = "new_runner_experiment"
21
22     """The path in which Experiment Runner will create a folder with the
23     name 'self.name', in order to store the
24     results from this experiment. (Path does not need to exist - it will be
25     created if necessary.)
26     Output path defaults to the config file's path, inside the folder '

```

```

25 experiments'"""
26 results_output_path: Path = ROOT_DIR / 'experiments'
27
28 """Experiment operation type. Unless you manually want to initiate each
29 run, use 'OperationType.AUTO'."""
30 operation_type: OperationType = OperationType.AUTO
31
32 """The time Experiment Runner will wait after a run completes.
33 This can be essential to accommodate for cooldown periods on some
34 systems."""
35 time_between_runs_in_ms: int = 1000
36
37 # Dynamic configurations can be one-time satisfied here before the
38 program takes the config as-is
39 # e.g. Setting some variable based on some criteria
40 def __init__(self):
41     """Executes immediately after program start, on config load"""
42
43     EventSubscriptionController.subscribe_to_multiple_events([
44         (RunnerEvents.BEFORE_EXPERIMENT, self.before_experiment),
45         (RunnerEvents.BEFORE_RUN, self.before_run),
46         (RunnerEvents.START_RUN, self.start_run),
47         (RunnerEvents.START_MEASUREMENT, self.start_measurement),
48         (RunnerEvents.INTERACT, self.interact),
49         (RunnerEvents.STOP_MEASUREMENT, self.stop_measurement),
50         (RunnerEvents.STOP_RUN, self.stop_run),
51         (RunnerEvents.POPULATE_RUN_DATA, self.populate_run_data),
52         (RunnerEvents.AFTER_EXPERIMENT, self.after_experiment)
53     ])
54     self.run_table_model = None # Initialized later
55
56     output.console_log("Custom config loaded")
57
58 def create_run_table_model(self) -> RunTableModel:
59     """Create and return the run_table model here. A run_table is a List
60     (rows) of tuples (columns),
61     representing each run performed"""
62     factor1 = FactorModel("fib_type", ['iter', 'mem', 'rec'])
63     factor2 = FactorModel("problem_size", [10, 35, 40, 5000, 10000])
64     self.run_table_model = RunTableModel(
65         factors=[factor1, factor2],
66         exclude_combinations=[
67             {factor2: [10]}, # all runs having treatment "10" will be
68             excluded
69             {factor1: ['rec'], factor2: [5000, 10000]},
70             {factor1: ['mem', 'iter'], factor2: [35, 40]}, # all runs
71             having combinations of ("mem", "iter") with ("35", "40") will be
72             excluded.
73         ],
74         repetitions = 10,
75         data_columns=["energy", "runtime", "memory"]
76     )
77     return self.run_table_model
78
79 def before_experiment(self) -> None:
80     """Perform any activity required before starting the experiment here
81     Invoked only once during the lifetime of the program."""
82     pass
83
84 def before_run(self) -> None:
85     """Perform any activity required before starting a run.
86     No context is available here as the run is not yet active (BEFORE

```

```

79     RUN)"""
80     pass
81
82     def start_run(self, context: RunnerContext) -> None:
83         """Perform any activity required for starting the run here.
84         For example, starting the target system to measure.
85         Activities after starting the run should also be performed here."""
86         pass
87
88     def start_measurement(self, context: RunnerContext) -> None:
89         """Perform any activity required for starting measurements."""
90         fib_type = context.execute_run["fib_type"]
91         problem_size = context.execute_run["problem_size"]
92
93         self.profiler = EnergiBridge(target_program=f"python examples/hello-
94         world-fibonacci/fibonacci_{fib_type}.py {problem_size}",
95         out_file=context.run_dir / "
96         energibridge.csv")
97
98         self.profiler.start()
99
100     def interact(self, context: RunnerContext) -> None:
101         """Perform any interaction with the running target system here, or
102         block here until the target finishes."""
103         pass
104
105     def stop_measurement(self, context: RunnerContext) -> None:
106         """Perform any activity here required for stopping measurements."""
107         stdout = self.profiler.stop(wait=True)
108
109     def stop_run(self, context: RunnerContext) -> None:
110         """Perform any activity here required for stopping the run.
111         Activities after stopping the run should also be performed here."""
112         pass
113
114     def populate_run_data(self, context: RunnerContext) -> Optional[Dict[str
115     , Any]]:
116         """Parse and process any measurement data here.
117         You can also store the raw measurement data under 'context.run_dir'
118         Returns a dictionary with keys 'self.run_table_model.data_columns'
119         and their values populated"""
120
121         eb_log, eb_summary = self.profiler.parse_log(self.profiler.logfile,
122         self.profiler.
123         summary_logfile)
124
125         return {"energy": list(eb_log["PACKAGE_ENERGY (J)"].values())[-1] -
126         list(eb_log["PACKAGE_ENERGY (J)"].values())[0],
127         "runtime": eb_summary["runtime_seconds"],
128         "memory": max(eb_log["USED_MEMORY"].values())}
129
130     def after_experiment(self) -> None:
131         """Perform any activity required after stopping the experiment here
132         Invoked only once during the lifetime of the program."""
133         pass
134
135 # ===== DO NOT ALTER BELOW THIS LINE
136 =====
137 experiment_path: Path = None

```

Listing 2: The RunnerConfig.py file.

Table 3: Example of the run_table.csv file generated by Experiment Runner.

__run_id	__done	fib_type	problem_size	energy	runtime	memory
run_0_repetition_0	DONE	iter	5000	31.23	1.40	10995630080
run_1_repetition_0	DONE	iter	10000	149.01	6.40	10972413952
run_2_repetition_0	DONE	mem	5000	3.41	0.2	10961080320
run_3_repetition_0	DONE	mem	10000	6.88	0.40	10972614656
run_4_repetition_0	DONE	rec	35	69.91	3.00	10978025472

After running the experiment with ER using the RunnerConfig.py file above, the run_table.csv file is generated. Table 3 shows a snippet of the data that can be found in the run_table.csv. The first two columns (*__run_id* and *__done*) are generated automatically by ER for every experiment, regardless of the configuration. The remaining columns are all user-defined and they can be divided into two categories: Factors (*fib_type* and *problem_size*) and Data Columns (*energy*, *runtime*, and *memory*).

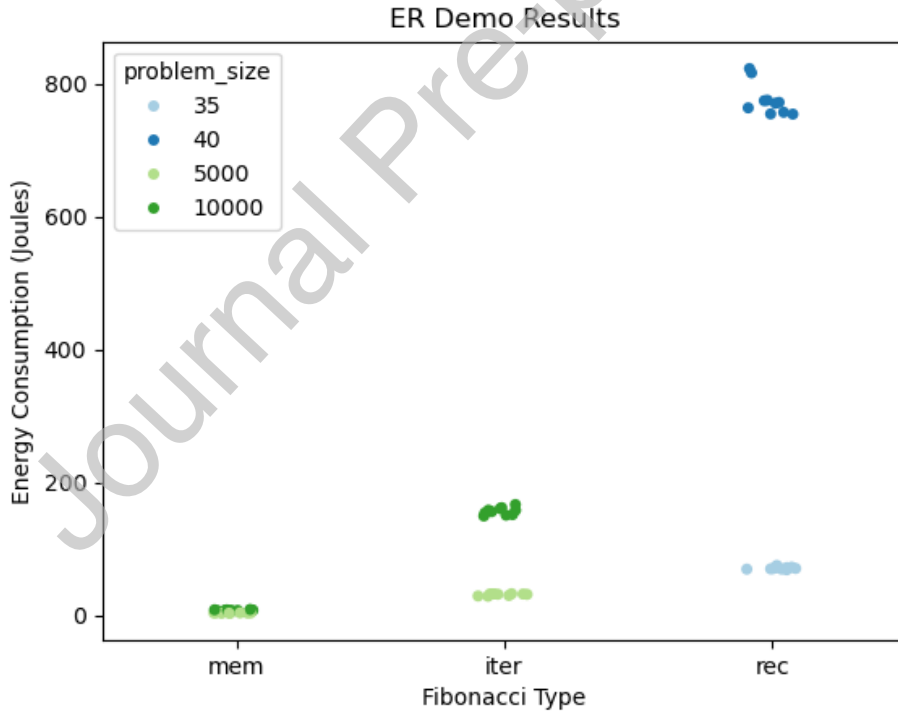
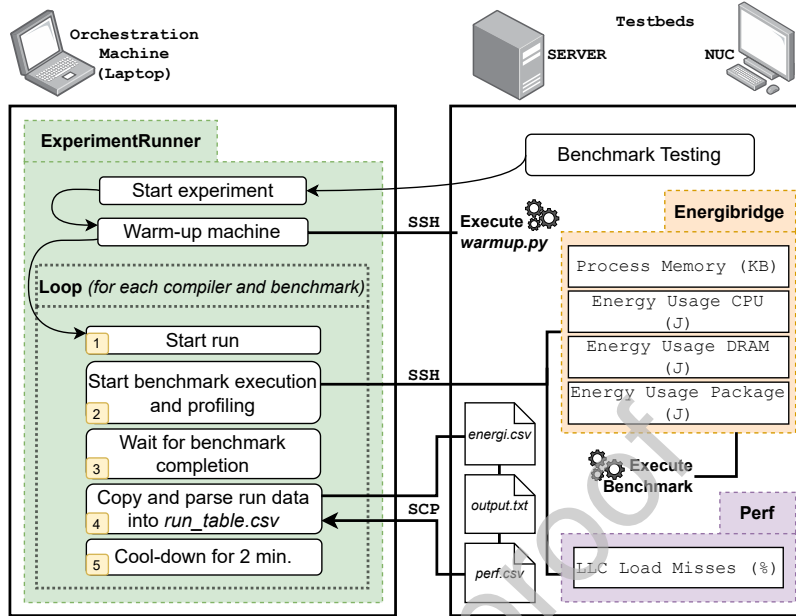


Figure 3: Fibonacci experiment energy results.

Figure 3 shows the results of the Fibonacci experiment regarding energy consumption. We notice in Figure 3 that *mem* performs the best in terms of

Figure 4: Experimental Setting of Stoico *et al.* [3]

energy consumption out of all the implementations, regardless of the problem size.

3.2. Recent studies based on ER

The sections showcase three examples of how ER can be used to orchestrate experiments involving different types of applications.

Stoico *et al.* [3] quantifies the impact of compilation on the energy efficiency and performance of Python code, which is widely used due to its extensive libraries and user-friendliness. However, Python typically exhibits lower energy efficiency and performance than compiled languages. Existing studies often ignore factors that can skew measurements, such as testbed and code characteristics. A common approach to enhancing the efficiency of Python using compilation. The authors evaluate the efficiency of compiled Python code across seven benchmarks using eight compilers. Figure 4 shows the experimental setting of the study. The experiment involves two machines: an orchestration machine (a laptop) and two testbeds (a server and an Next Unit of Computing device). The laptop runs ER and stores the measurements, which are taken on the testbeds during code execution. The operations performed with ER are illustrated in the green box and listed in their execution order. Warm-up and cool-down operations and automatic control of the profiling phases are crucial to performing unbiased experiments on soft-

ware energy efficiency and performance. The study compares interpreted and compiled Python code according to their energy usage and performance. The performance are evaluated according to the execution time, memory usage, and Last-Level Cache (LLC) miss rate during a run. Energy usage, execution time, and memory usage are measured using EnergiBridge [9], while the LLC miss rate using perf [15], a performance analyzer utility for Linux systems. Results show that the Codon, PyPy, and Numba compilers can optimize the energy efficiency and execution time of interpreted Python code by more than 85%. Compilation also has a positive effect on memory usage, but the extent of the optimization is highly dependent on code characteristics.

ER is used by Nguyen *et al.* [4] to experiment on Large Language Models (LLMs). The paper compares the energy usage of fetching content from an LLM locally (on-device) or from a remote server. While on-device LLMs offer advantages like improved privacy, running an LLM locally can significantly impact the energy consumption of a device. The work aims to inform software developers about the energy consumed when fetching content generated by LLMs in various scenarios, thus assisting them in optimizing the energy usage of client devices. The experiment involves seven LLMs, each one deployed on a laptop and a remote server. The laptop represents the on-device case, while the remote case is run on the server. The LLMs are asked to give information on a random topic selected among the most viewed pages on Wikipedia. The authors measure the energy usage, execution time, CPU, GPU, and memory usage of each local and remote request. ER runs on the laptop and orchestrates the experiment on both on-device and remote settings. ER coordinates the runs and starts/stops the profiling tools on the laptop. The tools measure the energy usage and the performance of each fetching request. The profiling tools used include psutil [16], a Python library that retrieves CPU and memory usage of running processes, powermetrics [7] to track GPU usage, and codecarbon [10] to monitor the energy usage of a request. Using on-device inference consumes 4 to 9 times more energy than retrieving content from the server. The execution time is directly related to energy consumption, with local inference being 2 to 3 times higher than remote fetching. An increase in CPU usage leads to lower energy consumption in all cases. This trend may be attributed to how the computing resources are used by LLMs, which shows a significant reliance on GPU usage. Consequently, in this experiment, GPU usage primarily influences energy consumption. The results did not show any evident pattern between energy and memory usage.

Nahrstedt *et al.* [17] compare the energy usage and performance of two well-known libraries for data manipulation: Pandas [18], written in Python, and Polars [19], a library written in Python with a core implemented in Rust

and designed for high-performance data analysis. Pandas stand out as a de-facto library for data manipulation, while Polars recently arose as a more efficient alternative to Pandas. This experiment aims to quantify whether and to what extent Polars improve the energy efficiency and performance of Pandas. The performance is quantified by comparing execution time, CPU, and memory usage. Both libraries are profiled while executing common operations on data frames called Data Analysis Tasks (DATs). Each DAT is a sequence of operations on data frames, such as ‘group by,’ ‘aggregation function for statistics,’ and ‘sort values,’ which group column values, calculate statistics, and sorts values, respectively. The authors consider 12 DATs, eight taken from the TPCB Benchmarking DATs and six self-written. The DATs are executed on dataframes of two sizes: large and small. Large dataframes have 60 and 15 million rows for TPCB Benchmarking DATs and self-written DATs, while small ones include 6 and 2 million rows. The experiment involves several runs, each one executing a combination of a library (*i.e.*, Pandas and Polars), a DAT, and a dataframe size (*i.e.*, large and small). The testbed consists of a single machine that runs ER and the benchmarks. The tools used by ER to track energy usage and performance are PowerJoular [13] a command line tool to profile energy usage, and ps [6], a Linux utility to profile the performance of active processes. The results show superior energy efficiency and performance of Polars. The difference is significant in all cases except for small dataframes exercised with DATs taken from the TPCB Benchmark.

4. Impact

So far, ER has had a meaningful impact on research and education by significantly streamlining the execution of measurement-based experiments. Before its introduction, students and researchers often had to build ad-hoc measurement pipelines for each experiment. This resulted in duplicated efforts, redundant scripting, and error-prone setups – especially in projects using similar technologies (*e.g.*, mobile front-ends or PostgreSQL back-ends). With ER, users now describe their experiment using a run table that mirrors the theoretical structure of the experimental design, defining factors, subjects, and treatments declaratively and letting the tool generate the experimental trials of interest automatically. This alignment between theory and implementation speeds up setup, reduces bugs, and makes the process more maintainable and reproducible.

The shift to ER has notably accelerated experimentation in the Green Lab course at Vrije Universiteit Amsterdam, where more than 100 students participate annually [20]. Previously, running experiments involved manual

control of treatments, creation, and debugging of custom scripts, as well as dealing with failures that could derail entire batches, sometimes even leading to significant data loss. Today, students and researchers can run their experiments automatically, pause and resume as needed, and cleanly aggregate results using standard formats (*e.g.*, CSV). As a result, the time to run a complete experiment has dropped significantly, and users can focus on interpretation and insight more.

ER has also become a central tool in a growing community of learners. It originated from lessons learned with Android Runner [21] and Robot Runner [5], earlier tools focused on specific domains like mobile and robotics and generalized those ideas into a flexible, domain-independent solution. Within the Software and Sustainability research group at Vrije Universiteit Amsterdam and the Green Lab ecosystem, ER has been used in at least 68 documented experiments by a wide range of learners: Green Lab students, Bachelor and Master thesis students, scientific assistants, and researchers. Students use the tool as a black-box orchestration engine, typically extending it with custom Python logic via its callbacks system. Assistants and contributors actively maintain and evolve the tool, contributing to new plugins, fixing bugs, and improving documentation. The community collaborates using regular open-source workflows (*e.g.*, pull requests, GitHub issues, code reviews), providing students with hands-on experience in collaborative software development.

5. Related Tools

Table 4 gives an overview of experiment orchestration tools with similar goals and features of ER (for comparison, we also include ER at the bottom of the table). We can observe that *all considered tools are targeting a specific type of system* and *five over eight tools are bound to specific families of metrics, i.e., energy, performance, CO₂*.

A relatively large subset of the considered tools target mobile applications. Specifically, PyAnaDroid, Android Runner, PETrA, and Greenminer are specific for Android applications and exploit their specificity for providing domain-specific functionalities. For example, PyAnaDroid allows users to instrument either Java or Kotlin source code in order to, *e.g.*, collect energy profiles at the level of source code block and for tracing the execution of Java/Kotlin methods in system’s logs; also, PyAnaDroid supports the integration with third-party source code analysis tools and Android-specific testing frameworks such as Espresso² and RERAN [22]. Similarly, Android

²<https://developer.android.com/training/testing/espresso>

Runner exploits its specificity to the Android platform to enable users to, for example, define experiments in a purely descriptive manner (*i.e.*, as a JSON file), without writing boilerplate code and have the tool automatically fetching, installing, executing, and uninstalling the apps targeted in the experiment. In this context, PETrA is similar to Android Runner, with the difference that PeTrA provides method-level measurement and full hardware-independence. Greenminer is different from other Android-specific tools since it comes with its own dedicated measurement hardware setup, which includes an Arduino board, an INA219 chip for power measurement, a Raspberry Pi, a USB hub, and smartphone, and a dedicated power supply. This configuration allows Greenminer to reach very high frequencies in terms of power sampling. Greenminer provides also a dedicated web service with data visualization features and it provides repository mining features for targeting versioned Android apps (this feature is particularly useful for longitudinal studies).

Table 4: Overview of tools related to Experiment Runner grouped by targeted systems (Ext. = Extensibility)

Tool name	Last update	Targeted systems	Metrics family	Ext.
PyAnaDroid ³ [23]	2025	Android apps	Independent	✓
Android Runner ⁴ [21]	2025	Android (web) apps	Independent	✓
PETrA ⁵ [24]	2021	Android apps	Energy	-
Greenminer ⁶ [25]	2018	Android apps	Energy	-
BenchExec ⁷ [26]	2025	Linux systems	Performance	✓
BenchCloud ⁸ [27]	2025	Distr. Linux systems	Performance	✓
Experiment-Impact-Tracker ⁹ [28]	2021	ML systems on Linux	Energy, CO ₂ , performance	✓
Robot Runner ¹⁰ [5]	2023	ROS systems	Independent	✓
ER¹¹	2025	Independent	Independent	✓

BenchExec and BenchCloud target exclusively Linux systems. Unlike ER, BenchExec focuses on single node experiments and is specialized in measuring performance-related aspects of the measured Linux system. BenchExec provides also features for the automatic generation of interactive tables and plots for inspecting collected measures and the integration of 3rd-party measurement scripts. BenchExec has a strong focus on collecting reliable measures and cores isolation; it is able to reliably measure and limit resource usage of the benchmarked Linux program, even if it spawns sub-processes (via `cgroups`). Examples of metrics supported out-of-the-box by BenchExec include CPU time, wall time, and peak memory consumption. Differently from ER, BenchExec does not support the automatic repetition of experimental runs for mitigating potential nondeterministic effects due to the in-

intrinsic volatility of certain physical phenomena, such as energy consumption. BenchCloud targets large-scale distributed systems composed of hundreds of Linux nodes. Internally, BenchCloud makes use of BenchExec for collecting performance measures from individual nodes. BenchCloud is designed to support elastic and scalable job distribution across hundreds of nodes, thus allowing users to perform large-scale experiments on distributed and heterogeneous computing environments. Similarly to ER and many other tools studied in this section, BenchCloud takes as input a configuration file which guides the orchestration of the whole benchmark; a BenchCloud configuration includes the program to be measured, the benchmark to be executed, the list of files to be retrieved after the experiment, and other experiment-specific parameters, such as the required CPU model of the nodes being measured and the computational limits for each run of the benchmark in terms of memory, time, and CPU cores. BenchCloud comes with a dedicated Web interface for managing experimental runs.

Experiment-Impact-Tracker is specialized on measuring performance and energy aspects of Machine Learning (ML) systems running on Linux machines. One of the most interesting features of this tool is that it is directly integrated with the notion of carbon intensity (via the IP address of the measured machine), allowing it to quantify the environmental footprint of the measured ML system in terms of CO₂ emissions. In addition, Experiment-Impact-Tracker provides a full snapshot of the main technical characteristics of the system under measurement in terms of, *e.g.*, all installed Python packages, CPU/GPU usage, CPU frequency, disk I/O speed. Experiment-Impact-Tracker also provides a mechanism to generate a CO₂ impact statement with the “social cost” of the measured ML system and a dedicated leaderboard to promote responsible research practices on energy-efficient ML algorithms.

Finally, it is worth mentioning Robot Runner, our tool for orchestrating experiments targeting robotic systems based on the ROS communication middleware. As already discussed in Section 4, ER builds on the lessons learned while we designed and developed Robot Runner in 2021. Both tools follow a plugin-based architecture (for extensibility purposes) and are completely independent from the metrics collected at runtime from the system being executed. In this context, ER can be considered as a domain-specific evolution of Robot Runner, designed to operate on any software system, regardless of its underlying technologies, deployment configurations, or distributed architecture. Moreover, ER provides the following new features with respect to Robot Runner: (i) an enhanced API for the programmatic creation of run tables, (ii) a generalization of the callbacks for providing custom business logic at specific phases of the experiment, (iii) a self-measurement configuration parameter which allows users to measure the energy consump-

tion of the experiment orchestration itself, this is particularly advised for academic studies since it is more and more important to transparently report the environmental footprint of performed experiments, (iv) a completely network-agnostic orchestration mechanism which allows users to run fully distributed experiments involving multiple heterogeneous physical nodes, and (v) a set of completely new plugins for profiling energy and resource consumption metrics for GPUs, multiple operating systems (including macOS and Windows), new hardware power profilers, and several new software-based energy profilers like CodeCarbon, EnergiBridge, and powerletrics.

6. Conclusions and Future Work

This paper presented Experiment Runner (ER), a Python framework for the (semi-) automatic orchestration of measurement-based experiments targeting software systems. Since its creation in 2022, ER has been used in tens of empirical studies in the area of Software Engineering, specially in studies targeting the energy efficiency of various types of software systems, such as microservice-based systems, high-performance computing, large language models, software libraries, ML-enabled systems, Internet of Things, web applications, etc. The source code of ER is available on GitHub¹², while its latest *stable* version at the time of writing is available on Zenodo [29].

As future work on ER, we are planning to (i) maintain ER's core orchestration components and develop a complete automated test suite for them, while fixing emerging bugs, (ii) develop additional plugins supporting (new) profilers, such as *macmon*¹³ for sudoless performance and energy monitoring for Apple Silicon processors, and (iii) carry out a controlled experiment aiming to systematically compare ER against existing experiment orchestration tools (see Section 5) in terms of, *e.g.*, reliability, effectiveness, usability, and system overhead.

Acknowledgements

We would like to thank all students and teaching assistants of the Green Lab course since the 2022/2023 academic year for having provided feedback and contributed to the development of Experiment Runner, with a special mention to Nikolaos Chalkiadakis for his contributions to the source code of the tool. We thank also the Computer Science department of the Vrije Universiteit Amsterdam for providing the organizational and financial support to

¹²<https://github.com/S2-group/experiment-runner>

¹³<https://github.com/vladkens/macmon>

maintain Experiment Runner. This project is partially supported by (i) the LETSGO Project, promoted by the Netherlands Enterprise Agency (Rijksdienst voor Ondernemend Nederland) and (ii) the European Union’s Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No 871342 “uDEVOPS”.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] C. Wohlin, P. Runeson, M. Höst, M. Ohlsson, B. Regnell, A. Wesslén, *Experimentation in Software Engineering*, Computer Science, Springer, 2012.
- [2] A. Guldner, R. Bender, C. Calero, G. S. Fernando, M. Funke, J. Groger, L. M. Hilty, J. Hornschemeyer, G.-D. Hoffmann, D. Junger, T. Kennes, S. Kreten, P. Lago, F. Mai, I. Malavolta, J. Murach, K. Obergoker, B. Schmidt, A. Tarara, J. De Veugh-Geiss, S. Weber, M. Westing, V. Wohlgemuth, S. Naumann, Development and evaluation of a reference measurement model for assessing the resource and energy efficiency of software products and components-green software measurement model (gsmm), *Future Generation Computer Systems* 155 (2024) 402–418.
- [3] V. Stoico, A. C. Dragomir, P. Lago, An empirical study on the performance and energy usage of compiled python code, in: *Proceedings of the 11th International Conference on Evaluation and Assessment on Software Engineering (EASE)*, IEEE, 2025, p. To appear.
- [4] V. Nguyen, V. Dhopate, H. Huynh, H. Bouhlal, A. Annengala, G. Scoccia, M. Martinez, V. Stoico, I. Malavolta, On-device or remote? on the energy efficiency of fetching llm-generated content, in: *4th IEEE/ACM International Conference on AI Engineering - Software Engineering for AI, CAIN 2025*, Lisbon, Portugal, April 27-28, 2025, IEEE, 2025, p. To appear.
- [5] S. Swanborn, I. Malavolta, Robot runner: A tool for automatically executing experiments on robotics software, in: *Proceedings of the*

- ACM/IEEE 43rd International Conference on Software Engineering, ACM, 2021, pp. 33–36.
- [6] The Linux man-pages project, ps(1) - report a snapshot of the current processes, <https://man7.org/linux/man-pages/man1/ps.1.html>, available on most Unix-like operating systems. Accessed via man ps (2025).
 - [7] Apple Inc., powermetrics, <https://firefox-source-docs.mozilla.org/performance/powermetrics.html>, accessed: 2025-05-16 (2025).
 - [8] WattsUp, Watts up? pro power monitor, <https://github.com/isaaclino/wattsup>, accessed: 2025-05-16.
 - [9] J. Sallou, L. Cruz, T. Durieux, Energibridge: Empowering software sustainability through cross-platform energy measurement (2023). arXiv: 2312.13897.
 - [10] V. Schmidt, K. Goyal, A. Joshi, B. Feld, L. Conell, N. Laskaris, D. Blank, J. Wilson, S. Friedler, S. Luccioni, CodeCarbon: Estimate and Track Carbon Emissions from Machine Learning Computing (2021). doi:10.5281/zenodo.4658424.
 - [11] D. Hoffmann, A. Tarara, Powerletrics: An open-source framework for power and energy metrics for linux, <https://github.com/green-kernel/powerletrics> (2025).
 - [12] Pico Technology Ltd., PicoLog CM3 Current Data Logger, Pico Technology Ltd., St Neots, United Kingdom (2015).
 - [13] A. Nouredine, Powerjoular: Real-time power consumption monitoring tool, <https://github.com/joular/powerjoular>, version 1.0.5. Accessed: 2025-05-16 (2024).
 - [14] NVIDIA Corporation, NVIDIA Management Library (NVML) Documentation, NVIDIA Corporation, Santa Clara, CA (2024).
 - [15] I. Molnár, Performance analysis tools for linux, <https://man7.org/linux/man-pages/man1/perf.1.html>, version 6.8.12. Accessed: 2025-05-16 (2024).
 - [16] G. Rodola, psutil: Cross-platform process and system utilities for python, <https://psutil.readthedocs.io>, version 7.0.1. Accessed: 2025-05-16 (2025).

- [17] F. Nahrstedt, M. Karmouche, K. Bargieł, P. Banijamali, A. Nalini Pradeep Kumar, I. Malavolta, An empirical study on the energy usage and performance of pandas and polars data analysis python libraries, in: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, 2024, pp. 58–68.
- [18] Wes McKinney, Data Structures for Statistical Computing in Python, in: Stéfan van der Walt, Jarrod Millman (Eds.), Proceedings of the 9th Python in Science Conference, 2010, pp. 56 – 61. doi:10.25080/Majora-92bf1922-00a.
- [19] Ritchie Vink and Polars Contributors, Polars: Dataframes powered by a multithreaded rust query engine and apache arrow, <https://github.com/pola-rs/polars>, version 0.19.0 (2020).
- [20] I. Malavolta, V. Stoico, P. Lago, Ten years of teaching empirical software engineering in the context of energy-efficient software, in: Handbook on Teaching Empirical Software Engineering, Springer, 2024, pp. 209–253.
- [21] I. Malavolta, E. M. Grua, C.-Y. Lam, R. de Vries, F. Tan, E. Zielinski, M. Peters, L. Kaandorp, A framework for the automatic execution of measurement-based experiments on android devices, in: 35th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW '20), ACM, 2020, pp. 61–66.
- [22] L. Gomez, I. Neamtii, T. Azim, T. Millstein, Reran: Timing-and touch-sensitive record and replay for android, in: 2013 35th international conference on software engineering (ICSE), IEEE, 2013, pp. 72–81.
- [23] R. Rua, J. Saraiva, Pyanadroid: A fully-customizable execution pipeline for benchmarking android applications, in: 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2023, pp. 586–591.
- [24] D. Di Nucci, F. Palomba, A. Prota, A. Panichella, A. Zaidman, A. De Lucia, Software-based energy profiling of android apps: Simple, efficient and reliable?, in: 2017 IEEE 24th international conference on software analysis, evolution and reengineering (SANER), IEEE, 2017, pp. 103–114.
- [25] A. Hindle, A. Wilson, K. Rasmussen, E. J. Barlow, J. C. Campbell, S. Romansky, Greenminer: A hardware based mining software repositories software energy consumption framework, in: Proceedings of the 11th working conference on mining software repositories, 2014, pp. 12–21.

- [26] D. Beyer, S. Löwe, P. Wendler, Reliable benchmarking: requirements and solutions, *International Journal on Software Tools for Technology Transfer* 21 (1) (2019) 1–29.
- [27] D. Beyer, P.-C. Chien, M. Jankola, Benchcloud: A platform for scalable performance benchmarking, in: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 2386–2389.
- [28] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, J. Pineau, Towards the systematic reporting of the energy and carbon footprints of machine learning, *Journal of Machine Learning Research* 21 (248) (2020) 1–43.
- [29] Software and Sustainability research group (S2), Experiment Runner (May 2025). doi:10.5281/zenodo.15430329.